Ex 8: Deploying a Fine-Tuned Language Model for Customer Support with Evaluation (BLEU & ROUGE)

Learning Objective:

Develop a Python-based application to fine-tune and deploy a language model for customer support using platforms like Hugging Face Spaces or Streamlit, and evaluate its performance on the entire test dataset using BLEU and ROUGE metrics.

Steps:

1. **Import Required Libraries**
   Import Transformers, Datasets, Torch, and evaluation libraries (BLEU, ROUGE).
2. **Load Dataset**
   Use a Customer Service dataset or Bank FAQ dataset containing queries and responses.
3. **Data Preprocessing**
   - Clean and normalize text
   - Prepare input-output pairs (query → response)
4. **Split Dataset**
   Divide into training and testing datasets.
5. **Load Pre-trained Model**
   Load models such as T5 or BART from Hugging Face.
6. **Fine-Tune the Model**
   Train the model using the training dataset.
7. **Save the Fine-Tuned Model**
   Store trained model and tokenizer.
8. **Build Inference Pipeline**
   Create a function to generate responses for input queries.
9. **Generate Predictions on Test Dataset**
   - Pass all test queries to the model
   - Generate responses for each query
10. **Compute BLEU Scores (BLEU-1 to BLEU-4)**

- Measure n-gram precision between generated and reference responses
- Compute BLEU-1, BLEU-2, BLEU-3, and BLEU-4 for each sample

11. **Compute ROUGE Scores**

- ROUGE-1 → unigram recall
- ROUGE-2 → bigram recall
- ROUGE-L for sequence similarity

12. **Aggregate Evaluation Metrics**

- Calculate average scores across all test samples
- Prepare overall performance summary

13. **Display Evaluation Results**

Metric	**Score**
BLEU-1	
BLEU-2	
BLEU-3	
BLEU-4	
ROUGE-1	
ROUGE-2	
ROUGE-L	

14. **Develop User Interface**

- Use Streamlit for web app
- OR deploy using Hugging Face Spaces

15. **Deploy the Application**

- Host the model for real-time customer query handling

16. **Test with Sample Queries**
    Validate the deployed system with real-time user inputs.

17. **Display Output**
    Show generated responses in the interface.

CODE :

```
from transformers import T5Tokenizer, T5ForConditionalGeneration
from nltk.translate.bleu_score import sentence_bleu
from rouge_score import rouge_scorer

model = T5ForConditionalGeneration.from_pretrained("t5-small")
tokenizer = T5Tokenizer.from_pretrained("t5-small")

data = {
    "How to reset password?": "Click on forgot password and follow steps",
    "What is account balance?": "Check your banking app or statement"
}

query = input("Enter your question: ")

input_ids = tokenizer(query, return_tensors="pt").input_ids
output = model.generate(input_ids, max_length=50)

response = tokenizer.decode(output[0], skip_special_tokens=True)

reference = data.get(query, "")

if reference:
    bleu = sentence_bleu([reference.split()], response.split())
else:
    bleu = 0

scorer = rouge_scorer.RougeScorer(['rouge1'], use_stemmer=True)
```

```
if reference:
    rouge = scorer.score(reference, response)
    rouge_score_value = rouge["rouge1"].fmeasure
else:
    rouge_score_value = 0

print("\nGenerated Response:", response)
print("Reference Answer:", reference)
print("BLEU Score:", round(bleu, 2))
print("ROUGE-1 Score:", round(rouge_score_value, 2))
```

OUTPUT :

```
 Enter your question: How to reset password?

Generated Response: Wie reset password password reset reset password?
Reference Answer: Click on forgot password and follow steps
BLEU Score: 0.0
ROUGE-1 Score: 0.14
```

Learning Outcome:

Students will be able to fine-tune, evaluate, and deploy a language model for customer support systems. They will gain practical experience in computing **BLEU and ROUGE scores**, enabling proper performance assessment of NLP models in real-world applications.